



Pasted markdown.md  
File

Получих файла с резюмето на Python прогреса ти.

Pasted markdown

Това е много добро “continuation summary” и реално вече си преминал от:

Python



Run

```
"уча синтаксис"
```

към:

Python



Run

```
"мисля в логика и workflow"
```

Най-важното в момента е, че:

- започваш да мислиш за state
- мислиш за validation
- мислиш за workflow rules
- комбинираш условия
- разбираш boundary logic
- разбираш processing flow

Това е огромната разлика между:

Python



Run

```
beginner who memorizes
```

и:

Python



Run

```
developer mindset foundation
```

Особено важно е това:

Python



Run

```
if "admin" not in roles and len(roles) < 5:
```

Защото тук вече:

- имаш collection reasoning
- validation reasoning
- business logic thinking

а не просто “учене на Python”.

---

Следващата правилна стъпка за теб наистина е:

## Етап 7 — Part 4

### Low-Hint Boolean Workflow Challenges

Точно сега НЕ ти трябва:

- OOP
- frameworks
- APIs
- Django
- FastAPI
- AI agents

А трябва:

- автоматизация на логиката
- по-бързо condition reasoning
- по-малко подсказки
- по-сложни workflow проверки
- state evaluation

Това е правилната основа за:

- backend мислене
  - automation
  - AI workflows
  - validation systems
  - agent logic
  - production reasoning
- 

Между другото, това резюме е много добро и като “import profile” за нов чат. Реално новият чат ще разбере:

- какво вече знаеш
- как мислиш

- какво НЕ трябва да се повтаря
- къде са слабите ти места
- каква е правилната следваща стъпка

Особено ценна е секцията:

</> Python

CURRENT WEAK AREAS

Run

защото тя показва:

- че вече не правиш syntax errors основно
- а precision / reasoning / speed issues

което е следващото ниво. Sources

какво следва сега?

Сега следва много важен преход:

От:

</> Python

"разбирам отделни конструкции"

Run

Към:

</> Python

"изграждам processing systems"

Run

Това означава:

## PHASE 1 — Low-Hint Workflow Challenges

Това е директното продължение.

Ще започнеш задачи тип:

- inventory systems
- validation flows

- mini backend logic
- access control
- filtering systems
- cart validation
- state tracking

НО:

- с по-малко подсказки
- с повече самостоятелно мислене
- с по-дълги condition chains

---

## Какво ще тренираш реално

### 1. Boolean speed

По-бързо разчитане на:

Python

```
and  
or  
not  
in  
len()
```



Run

и комбинирането им.

---

### 2. Condition architecture

Примерно:

Python

```
if user in users and role == "admin" and len(cart) > 0:
```



Run

Да започнеш да:

- виждаш condition blocks
- четеш logic flow
- разбираш validation pipelines

---

### 3. State reasoning

Това е огромната тема.

Да започнеш да мислиш:

Python



Run

"Какво е текущото състояние на системата?"

а не:

Python



Run

"Какъв е синтаксисът?"

---

**След ТОВА идва:**

## Етап 8 — List Processing Automation

Там вече ще започнеш:

- по-големи traversal задачи
- multi-condition filtering
- transformation logic
- processing pipelines

И тогава мозъкът ти започва да влиза в:

- backend thinking
- data processing
- automation thinking

---

## След Етап 8

Тогава вече логично идват:

### Functions

Защото тогава ще почувстваш проблема:

Python



Run

"Повтарям едни и същи processing patterns"

и ще видиш ЗАЩО functions съществуват.

---

# След functions

Тогава:

- dictionaries
- real workflows
- files
- JSON
- APIs
- automation scripts

---

## И най-важното:

Ти в момента си на ПРАВИЛНИЯ етап.

Много хора:

- скачат към frameworks
- копират AI generated code
- правят tutorial hell
- не могат да мислят processing logic

При теб вече има:

- structural thinking
- debugging instinct
- workflow reasoning
- boundary awareness
- validation thinking

Това са реалните foundations.

Особено важно:

ти НЕ учиш Python като “език”.

Ти започваш да учиш:

Python



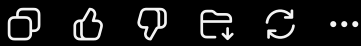
Run

```
computational reasoning
```

И това е правилната посока за:

- AI-assisted development
- automation
- backend

- tooling
- agents
- real-world coding workflows.



в предишния чат стигнахме както виждаш от .md до  
Part 4 — Low-Hint Boolean Workflow Challenges

Да — и това е абсолютно логичното място, до което да стигнеш след всичко от  
резюмето.  Pasted markdown

Реално ти вече си преминал:

## Phase Transition

От:

 Python



 Run

`guided syntax practice`

Към:

 Python



 Run

`autonomous logic construction`

И точно затова следва:

## Part 4 — Low-Hint Boolean Workflow Challenges

### Какво означава “Low-Hint”

До момента:

- задачите са били структурирани
- имало е ясна посока
- pattern-ът е бил видим

Сега започваш:

- сам да откриваш pattern-a
- сам да строиш condition-a
- сам да решаваш:
  - AND или OR
  - кога len()
  - кога in
  - кога flag
  - кога break
  - кога traversal

## Това е много важен етап

Тук мозъкът започва да прави:

Python



Run

```
problem decomposition
```

Вместо:

Python



Run

```
syntax recall
```

## Как ще изглеждат задачите

Не:

Python



Run

```
"направи if с and"
```

А:

Python



Run

```
"валидирай система за поръчки"  
"провери достъп"  
"открий конфликт"  
"филтрирай проблемни записи"
```

и ти сам трябва да решиш:



- каква логика да използваш
- как да комбинираш условията
- какъв processing flow трябва

---

## Това е преход към REAL coding thinking

Тук започва:

- backend instinct
- automation instinct
- debugging instinct
- workflow reasoning

---

## Най-важното

Твоят roadmap в момента е МНОГО правилен.

Защото:

- не скачаш към frameworks
- не прескачаш fundamentals
- не зависиш изцяло от AI autocomplete

А изграждаш:

</> Python



▶ Run

```
reasoning engine
```

---

## След като стабилизираш Part 4

Тогава идва:

## Етап 8 — Processing Automation

Там вече:

- traversal става по-голям
- condition layering става по-сериозен
- filtering pipelines стават по-реални
- започва “data processing thinking”

И тогава ще започнеш да усещаш:

- защо functions са нужни
- защо reusable logic е важен
- защо backend processing е architecture problem

Не просто syntax ... Sources

добре, хайде продължаваме със заданията до където  
бях стигнал

## Part 4 — Low-Hint Boolean Workflow Challenges

### Challenge 1 — Access Validation System

</> Python



▶ Run

```
users = ["ivan", "maria", "petar"]
roles = ["editor", "support"]

username = "ivan"

# Проверка:
# 1. username трябва да е в users
# 2. "admin" НЕ трябва да е в roles
# 3. users трябва да са поне 3

# Ако всички условия са изпълнени:
# print("Limited access granted")
```

### Challenge 2 — Product Availability

</> Python



▶ Run

```
products = ["mouse", "keyboard", "monitor"]
cart = ["keyboard", "monitor"]

# Проверка:
# 1. "keyboard" трябва да е в products
# 2. cart трябва да съдържа поне 2 продукта
# 3. "printer" НЕ трябва да е в cart

# Ако условията са изпълнени:
# print("Order ready")
```

## Challenge 3 — Delivery Validation

Python



Run

```
cities = ["Sofia", "Plovdiv", "Varna"]
active_drivers = ["Georgi", "Ivan"]

selected_city = "Sofia"

# Проверка:
# 1. selected_city трябва да е в cities
# 2. active_drivers трябва да са поне 2
# 3. "Blocked" НЕ трябва да е в cities

# Ако условията са изпълнени:
# print("Delivery available")
```

## Challenge 4 — Repetition Workflow

Python



Run

```
codes = ["A1", "B2"]

generated_codes = codes * 3

# Проверка:
# 1. generated_codes трябва да съдържа "A1"
# 2. generated_codes трябва да имат дължина над 5
# 3. "X9" НЕ трябва да съществува

# Ако условията са изпълнени:
# print("Code generation successful")
```

## Challenge 5 — Combined Inventory Logic

Python



Run

```
warehouse_a = ["monitor", "keyboard"]
warehouse_b = ["mouse", "chair"]

combined_inventory = warehouse_a + warehouse_b

# Проверка:
```

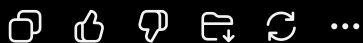
```
# 1. "chair" трябва да съществува
# 2. combined_inventory трябва да съдържа поне 4 продукта
# 3. "printer" НЕ трябва да съществува

# Ако условията са изпълнени:
# print("Inventory validated")
```

Не бързай.

Искам:

- самостоятелно condition building
- внимателно четене на constraints
- reasoning преди писане.



# Part 4 — Low-Hint Boolean Workflow Challenges

# Challenge 1 — Access Validation System

users = ["ivan", "maria", "petar"]

roles = ["editor", "support"]

username = "ivan"

if username in users and 'admin' not in roles and len(users)  
>=3:

print("Limited access granted")

# Challenge 2 — Product Availability

products = ["mouse", "keyboard", "monitor"]

Show more ▾

Отлично. И 5-те са правилни.

Най-важното: тук вече няма “случайно познаване”. Ти правилно си комбинирал:

Python



Run

```
in
not in
len()
and
+
*
```

Оценка:

</> Python



▶ Run

Challenge 1: правилно  
Challenge 2: правилно  
Challenge 3: правилно  
Challenge 4: правилно  
Challenge 5: правилно

Единствена дребна стилова забележка:

</> Python



▶ Run

```
len(users) >=3
```

по-чисто е:

</> Python



▶ Run

```
len(users) >= 3
```

Но това е само readability, не логическа грешка.

Следващото ниво трябва да е малко по-трудно: условия с `or` + смесена логика, защото досега всички са основно `and`.

Готов си за:

## Part 4B — Mixed AND / OR Workflow Challenges



добре, готов съм за Part 4B — Mixed AND / OR Workflow Challenges, искам само да знаеш че съм на Етап 7 от този roadmap, надявам се and, or и т.н. да не ме отклонява от уроците за List

Не — точно обратното.

Това Е част от Етап 7, не отклонение.

От screenshot-a roadmap-ът ти е:

</> Python



▶ Run

Етап 7: List операции (+, \*, in, len)

И реално:

- and
- or
- boolean composition
- validation logic

са естественото ПРОДЪЛЖЕНИЕ на:

</> Python



▶ Run

```
in
not in
len()
+
*
```

Защото тези операции сами по себе си са само инструменти.

Истинският coding layer идва когато започнеш да правиш:

</> Python



▶ Run

workflow decisions

Пример:

</> Python



▶ Run

```
if "admin" in roles and len(users) > 3:
```

Това все още е:

- list reasoning
- membership reasoning
- collection reasoning

НЕ е нова отделна тема.

---

Всъщност ти в момента правиш най-важната част от Етап 7:

## Operational Thinking

Тоест:

- използваш list operations
  - за да взимаш решения
  - върху collections/state/workflows
- 

Ако прескочим това и отидем директно на:

`</> Python`



 Run

```
slicing
functions
comprehensions
```

ще се получи:

- syntax without reasoning
- shallow understanding
- tutorial-style progression

А при теб целта е:

- processing mindset
  - workflow mindset
  - backend reasoning
- 

Така че спокойно:

**Ти НЕ си извън roadmap-а.**

Ти си:

**в дълбоката и правилна част на Етап 7.**

И честно казано:

това е мястото, където много хора се чупят,  
защото:

- могат syntax
- но не могат logic composition

А при теб това започва да се изгражда правилно.

---

Готов си.

**Part 4B — Mixed AND / OR Workflow Challenges**

Тук вече:

- conditions стават по-реалистични
- ще има:
  - AND
  - OR
  - mixed validation
  - multiple valid paths
  - fallback logic

И това е супер-важен transition stage.



очаквам заданията

## Part 4B — Mixed AND / OR Workflow Challenges

### Challenge 1 — VIP Access Logic

</> Python



▶ Run

```
users = ["ivan", "maria", "petar"]
roles = ["support"]

username = "ivan"

# Достъп се дава ако:
# 1. username е в users
# И
# 2. roles съдържа "admin"
# ИЛИ
# roles съдържа "support"

# Ако условията са изпълнени:
# print("VIP access granted")
```

### Challenge 2 — Shipping Validation

</> Python



▶ Run

```
products = ["mouse", "keyboard", "monitor"]
cart = ["keyboard"]

# Поръчката е валидна ако:
```



```
# 1. "keyboard" е в cart
# И
# 2. cart има поне 2 продукта
# ИЛИ
# "monitor" е в products

# Ако условията са изпълнени:
# print("Shipping approved")
```

## Challenge 3 — Multi-City Delivery

</> Python



▶ Run

```
cities = ["Sofia", "Plovdiv"]
drivers = ["Ivan"]

selected_city = "Varna"

# Доставка е възможна ако:
# 1. selected_city е в cities
# ИЛИ
# cities съдържа "Sofia"
#
# И
#
# 2. drivers има поне 1 шофьор

# Ако условията са изпълнени:
# print("Delivery started")
```

## Challenge 4 — Combined Inventory Rules

</> Python



▶ Run

```
warehouse_a = ["monitor"]
warehouse_b = ["mouse", "keyboard"]

combined = warehouse_a + warehouse_b

# Inventory е валиден ако:
# 1. combined съдържа "monitor"
# И
# 2. combined съдържа "printer"
# ИЛИ
# len(combined) >= 3
```

```
# Ако условията са изпълнени:  
# print("Inventory system ready")
```

## Challenge 5 — Repetition Access System

</> Python



▶ Run

```
codes = ["A1", "B2"]  
  
generated = codes * 2  
  
# Системата е успешна ако:  
# 1. generated съдържа "A1"  
# И  
# 2. generated НЕ съдържа "X9"  
# ИЛИ  
# len(generated) > 10  
  
# Ако условията са изпълнени:  
# print("System activated")
```

Тук вече искам:

- да мислиш за priority
- как Python чете условията
- кога `or` “спасява” condition-a

как се комбинират блокове логика.



# Part 4B — Mixed AND / OR Workflow Challenges

# Challenge 1 — VIP Access Logic

users = ["ivan", "maria", "petar"]

roles = ["support"]

username = "ivan"

if username in users and 'admin' in roles or 'support' in roles:

print("VIP access granted")

# Challenge 2 — Shipping Validation

products = ["mouse", "keyboard", "monitor"]

Show more ▾

Thought for 7s >

Много добро изпълнение. Кодът ще работи, но тук вече стигнахме до **ключовия проблем на Part 4B**:

`and` **има по-висок приоритет от** `or`

Python чете това:

Python



Run

```
A and B or C
```

като:

Python



Run

```
(A and B) or C
```

а НЕ като:

Python



Run

```
A and (B or C)
```

## Анализ

### Challenge 1

ТВОЯТ КОД:

Python



Run

```
if username in users and 'admin' in roles or 'support' in roles:
```

Python го чете така:

Python



Run

```
if (username in users and 'admin' in roles) or 'support' in roles:
```

Но задачата искаше:

</> Python



▶ Run

```
if username in users and ('admin' in roles or 'support' in roles):
```

Тук трябва скоби.

---

## Challenge 2

ТВОЯТ КОД:

</> Python



▶ Run

```
if 'keyboard' in cart and len(cart) >= 2 or 'monitor' in products:
```

Python го чете така:

</> Python



▶ Run

```
if ('keyboard' in cart and len(cart) >= 2) or 'monitor' in products:
```

Това отговаря на формулировката. ✓

---

## Challenge 3

ТВОЯТ КОД:

</> Python



▶ Run

```
if selected_city in cities or 'Sofia' in cities and len(drivers) >= 1:
```

Python го чете така:

</> Python



▶ Run

```
if selected_city in cities or ('Sofia' in cities and len(drivers) >= 1):
```

Но задачата искаше:

</> Python



▶ Run

```
if (selected_city in cities or 'Sofia' in cities) and len(drivers) >= 1:
```

Тук трябва скоби.

---

## Challenge 4

ТВОЯТ КОД:

</> Python



▶ Run

```
if 'monitor' in combined and 'printer' in combined or len(combined) >= 3:
```

Python го чете така:

</> Python



▶ Run

```
if ('monitor' in combined and 'printer' in combined) or len(combined) >= 3:
```

Това отговаря на формулировката. ✓

---

## Challenge 5

ТВОЯТ КОД:

</> Python



▶ Run

```
if 'A1' in generated and 'X9' not in generated or len(generated) > 10:
```

Python го чете така:

</> Python



▶ Run

```
if ('A1' in generated and 'X9' not in generated) or len(generated) > 10:
```

Това отговаря на формулировката. ✓

---

## Коригиран вариант

</> Python



▶ Run

```
# Challenge 1
if username in users and ('admin' in roles or 'support' in roles):
    print("VIP access granted")

# Challenge 3
if (selected_city in cities or 'Sofia' in cities) and len(drivers) >= 1:
    print("Delivery started")
```

Изводът: когато имаш смесено `and` + `or`, започвай да слагаш скоби, за да контролираш логиката. Това е точно правилният урок за този етап.

добре, може би ще трябва да направим repair drill по повод приоритизирането при and/or и скобите?

Да.

И честно казано — това е много правилен момент за repair drill.

Защо?

Защото ти НЕ бъркаш:

- `in`
- `len`
- `syntax`
- `list operations`

А вече стигна до:

## Logic grouping

Това е по-високо ниво.

## Реалният проблем

Ти четеш:

Python



Run

```
A and B or C
```

като:

Python



Run

```
A and (B or C)
```

Но Python чете:

Python



Run

```
(A and B) or C
```

защото:

```
</> Python
```



▶ Run

```
and
```

има по-висок приоритет от:

```
</> Python
```



▶ Run

```
or
```

---

## Това е нормален етап

Много важно:

това НЕ е beginner syntax issue.

Това е:

```
</> Python
```



▶ Run

```
logical grouping issue
```

което е:

- реална programming тема
- реален debugging source
- реален backend bug source

---

## И да — repair drill е правилният ход

Но repair drill-ът трябва да е:

- focused
- short
- intense
- only on grouping logic

НЕ нови теми.

---

# Repair Drill — AND / OR Priority

## Rule:

Python чете:

Python



Run

```
A and B or C
```

като:

Python



Run

```
(A and B) or C
```

## Repair Challenge 1

Как Python чете това?

Python



Run

```
if True or False and False:
```

Кое е вярното?

Python



Run

```
A) (True or False) and False  
B) True or (False and False)
```

И какъв е крайният резултат?

## Repair Challenge 2

Как Python чете това?

Python



Run

```
if False and True or True:
```

Кое е вярното?



Python



Run

```
A) (False and True) or True
B) False and (True or True)
```

И какъв е крайният резултат?

## Repair Challenge 3

Добави правилните скоби така, че:

Python



Run

```
or
```

да се evaluated ПЪРВО.

Python



Run

```
if "admin" in roles and "support" in roles or len(users) > 3:
```

## Repair Challenge 4

Добави правилните скоби така, че:

Python



Run

```
and
```

да се evaluated последно.

Python



Run

```
if "A1" in codes or "B2" in codes and len(codes) > 5:
```

## Repair Challenge 5

Python как чете това?

Python



Run

```
if False or True and False or True:
```

Напиши:

grouping  
• final result

Repair Challenge 1

B) True or (False and False)

#True

Repair Challenge 2

A) (False and True) or True

#True

Repair Challenge 3

if ("admin" in roles and "support" in roles) or len(users) > 3:

Repair Challenge 4

if ("A1" in codes or "B2" in codes) and len(codes) > 5:

Repair Challenge 5

Show more ▾

Много добре.

Тук вече се вижда, че repair drill-ът работи.

## Challenge 1

Ти:

</> Python



▶ Run

```
True or (False and False)
```

✓ правилно

</> Python



▶ Run

```
False and False → False
```

```
True or False → True
```

✓ краен резултат: True

## Challenge 2

Ти:

Python



Run

```
(False and True) or True
```

✓ правилно

Python



Run

```
False and True → False
```

```
False or True → True
```

✓ краен резултат: True

## Challenge 3

Ти:

Python



Run

```
if ("admin" in roles and "support" in roles) or len(users) > 3:
```



Тук задачата беше:

OR да се evaluated ПЪРВО

Но ти остави:

Python



Run

```
and
```

първо.

Трябва:

Python



Run

```
if "admin" in roles and ("support" in roles or len(users) > 3):
```

Защото така:

Python



Run

```
("support" in roles or len(users) > 3)
```

се evaluated първо.

## Challenge 4

Ти:

Python



Run

```
if ("A1" in codes or "B2" in codes) and len(codes) > 5:
```



правилно

Тук:

- `or` става първо
- `and` остава последно

Точно това искаше задачата.

## Challenge 5

Python



Run

```
if False or True and False or True:
```

Python чете:

Python



Run

```
False or (True and False) or True
```

защото:

Python



Run

```
and
```

има по-висок priority.

После:

Python



Run

```
True and False → False
```

става:

Python



Run

```
False or False or True
```

После:

Python



Run

```
False or False → False  
False or True → True
```



краен резултат:

Python



Run

```
True
```

## Извод

Тук вече:

- не учиш syntax
- а evaluation flow

Това е:

- debugging thinking
- backend reasoning
- condition parsing

И честно казано:

ти доста бързо хвана проблема със скобите.



тоест скобите дават приоритет дори на or

Точно така.

Скобите казват на Python:

</> Python



▶ Run

```
"изчисли това първо"
```

дори ако вътре има:

</> Python



▶ Run

```
or
```

## Без скоби

Python има собствен priority:

</> Python



▶ Run

```
1. ()  
2. and  
3. or
```

## Пример без скоби

</> Python



▶ Run

```
True or False and False
```

Python чете:

</> Python



▶ Run

```
True or (False and False)
```

защото:

</> Python



▶ Run

```
and
```

е по-силен от:

```
<> Python
```



▶ Run

```
or
```

## Но със скоби

```
<> Python
```



▶ Run

```
(True or False) and False
```

Сега Python е длъжен:

- първо да изчисли скобите
- независимо дали вътре има `or`

## Това е същото като в математиката

Например:

```
<> Python
```



▶ Run

```
2 + 3 * 4
```

става:

```
<> Python
```



▶ Run

```
2 + 12 = 14
```

Но:

```
<> Python
```



▶ Run

```
(2 + 3) * 4
```

става:

```
<> Python
```



▶ Run

5 \* 4 = 20

Скобите override-ват нормалния priority.

## Programming insight

Това е супер важна тема.

Защото в реални systems често имаш:

</> Python



▶ Run

```
if is_admin and has_access or is_owner:
```

И без скоби може да се получи:

- security bug
- validation bug
- wrong access logic

За това добрите developers често слагат скоби дори когато не са задължителни — за readability и safety.

имам навика когато правим repair drill той да е active coding drill, искам 3 задачи за active repair за and, or, ()

## Active Repair Drill — AND / OR / ()

### Repair Task 1 — Access Logic

Поправи condition-а така, че:

</> Python



▶ Run

```
or
```

да се evaluated ПЪРВО.

</> Python



▶ Run

```
users = ["ivan", "maria"]
roles = ["editor"]
```



```
if "ivan" in users and "admin" in roles or len(users) >= 2:  
    print("Access granted")
```

## Repair Task 2 — Delivery Workflow

Поправи condition-а така, че:

</> Python



▶ Run

```
and
```

да се evaluated ПОСЛЕДНО.

</> Python



▶ Run

```
cities = ["Sofia"]  
drivers = ["Ivan"]  
  
if "Varna" in cities or "Sofia" in cities and len(drivers) >= 1:  
    print("Delivery active")
```

## Repair Task 3 — Inventory Validation

Добави правилните скоби така, че логиката да стане:

</> Python



▶ Run

```
("monitor" in products OR "keyboard" in products)  
AND  
len(products) >= 3
```

Начален код:

</> Python

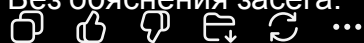


▶ Run

```
products = ["monitor", "mouse", "keyboard"]  
  
if "monitor" in products or "keyboard" in products and len(products) >= 3:  
    print("Inventory ready")
```

Само active coding.

Без обяснения засега.



```
users = ["ivan", "maria"]
roles = ["editor"]
```

```
if "ivan" in users and "admin" in roles or (len(users) >= 2):
    print("Access granted")
cities = ["Sofia"]
drivers = ["Ivan"]
```

```
if ("Varna" in cities or "Sofia" in cities) and len(drivers) >= 1:
    print("Delivery active")
```

Show more ▾

Repair drill-ът си свърши работата.

## Task 1

ТВОЯТ КОД:

Python



Run

```
if "ivan" in users and "admin" in roles or (len(users) >= 2):
```

✗ Не напълно.

Ти сложи скоби, но върху:

Python



Run

```
(len(users) >= 2)
```

а не върху:

Python



Run

```
or
```

Задачата беше:

or да се evaluated първо

Правилният repair:

</> Python



▶ Run

```
if "ivan" in users and ("admin" in roles or len(users) >= 2):
```

## Task 2

</> Python



▶ Run

```
if ("Varna" in cities or "Sofia" in cities) and len(drivers) >= 1:
```

✓ правилно

Тук:

- `or` става първо
- `and` остава последно

## Task 3

</> Python



▶ Run

```
if ("monitor" in products or "keyboard" in products) and len(products) >= 3:
```

✓ перфектно

Точно това беше target logic.

## Много важно наблюдение

Вече започваш да:

- виждаш logical blocks
- местиш evaluation priority
- контролираш processing flow

Това е реалният смисъл на скобите в programming logic.

## Следващият repair layer

Трябва още малко automation върху:

<> Python



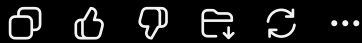
▶ Run

```
"Къде точно трябва да сложа скобите?"
```

защото в Task 1:

- разбра, че трябва скоба
- но grouping-ът още не е напълно автоматизиран

което е нормално за този етап.



днес е нов ден, какво е най-добре и полезно за мен да направим днес с оглед грешките ми и нуждата от repair, какво предлагаш за днешната ни сесия?

Най-полезното днес е къса repair сесия + контролирано връщане към Етап 7.

Предлагам така:

## Днешна сесия — 60–80 мин

### 1. Warm-up: Boolean grouping repair — 15 мин

Само `and / or / ()`.

Цел: да спреш да слагаш скоби “около едно условие” и да започнеш да ги слагаш около логически блок.

### 2. Active coding repair — 25 мин

Ще ти давам счупени conditions и ти ще ги поправяш.

Фокус:

<> Python



▶ Run

```
A and (B or C)
(A or B) and C
(A and B) or C
```

### 3. Етап 7 workflow задачи — 25 мин

Връщаме се към:

</> Python



▶ Run

```
+  
*  
in  
not in  
len()
```

но с правилно групиране на условия.

## 4. Mini test — 10 мин

5 задачи без подсказки, за да видим дали герайг-ът е стабилизирал.

За днес НЕ предлагам нова тема.

Не бих минавал към Етап 8 още, защото последната ти грешка е точно важна:

</> Python



▶ Run

```
if A and B or C:
```

Трябва да стане автоматично да питаш:

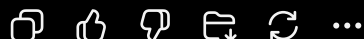
</> Python



▶ Run

```
"Коя част е логическият блок?"
```

Започваме с 3 warm-up задачи.



очаквам заданието..

## Днешна сесия — Active Repair Drill 1

Поправи само `if` условията. Не променяй списъците и `print()`.

### Task 1 — OR трябва да е първо

</> Python



▶ Run

```
users = ["ivan", "maria"]
roles = ["editor"]

if "ivan" in users and "admin" in roles or len(users) >= 2:
    print("Access granted")
```

Целевата логика е:

Python



Run

```
"ivan" in users AND ("admin" in roles OR len(users) >= 2)
```

## Task 2 — AND трябва да е първо

Python



Run

```
products = ["mouse", "keyboard"]
cart = ["keyboard"]

if "keyboard" in cart and len(cart) >= 2 or "mouse" in products:
    print("Order accepted")
```

Целевата логика е:

Python



Run

```
("keyboard" in cart AND len(cart) >= 2) OR "mouse" in products
```

## Task 3 — OR блок + len проверка

Python



Run

```
cities = ["Sofia", "Plovdiv"]
drivers = ["Ivan"]

if "Sofia" in cities or "Varna" in cities and len(drivers) >= 1:
    print("Delivery active")
```

Целевата логика е:

Python



Run

```
("Sofia" in cities OR "Varna" in cities) AND len(drivers) >= 1
```

## Task 4 — Membership alternatives

Python



Run

```
products = ["monitor", "mouse", "keyboard"]

if "monitor" in products or "keyboard" in products and "printer" not in products:
    print("Inventory valid")
```

Целевата логика е:

Python



Run

```
("monitor" in products OR "keyboard" in products) AND "printer" not in products
```

## Task 5 — Access fallback

Python



Run

```
codes = ["A1", "B2"]
generated = codes * 2

if "A1" in generated and "X9" not in generated or len(generated) > 10:
    print("System activated")
```

Целевата логика е:

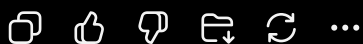
Python



Run

```
("A1" in generated AND "X9" not in generated) OR len(generated) > 10
```

Изпрати само коригираните 5 решения.



# Active Repair Drill 1

# Task 1 — OR трябва да е първо

users = ["ivan", "maria"]

roles = ["editor"]

```
if "ivan" in users and ("admin" in roles or len(users) >= 2):
```

```
print("Access granted")
# Task 2 — AND трябва да е първо
products = ["mouse", "keyboard"]
cart = ["keyboard"]

Show more ▾
```

Прав си. Това не беше добър repair drill, защото ти дадох почти готовото групиране.

Твоите 5 решения са правилни, но упражнението беше твърде подсказващо.

По-добрият формат за теб трябва да е:

Python



Run

1. Давам бизнес условие с думи.
2. Давам счупен if.
3. Ти сам решаваш кои условия са един логически блок.
4. После слагаш скобите.

Пример за правилен формат:

Python



Run

```
# Достъп има само ако потребителят е Ivan
# и има поне едно от следните:
# - admin роля
# - поне 2 потребителя в системата

if "ivan" in users and "admin" in roles or len(users) >= 2:
    print("Access granted")
```

Тогава ти сам трябва да видиш:

Python



Run

```
if "ivan" in users and ("admin" in roles or len(users) >= 2):
```

Да, така трябва да го правим занапред.



за начало, дай ми една задача само по новия метод за да тествам



Отлично. Ето това вече е истински repair drill.

Няма да ти казвам къде са скобите.

Ще ти дам:

- бизнес логиката
- счупения condition

Ти трябва да поправиш condition-а така, че да отговаря на описанието.

## Repair Task

Python



Run

```
users = ["ivan", "maria"]
roles = ["editor"]

# Достъп се дава само ако:
# 1. "ivan" е в users
# И
# 2. има поне едно от следните:
#    - "admin" е в roles
#    - roles съдържа поне 1 роля

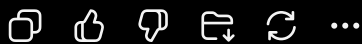
if "ivan" in users and "admin" in roles or len(roles) >= 1:
    print("Access granted")
```

Поправи само `if` условието.

Не обяснявай.

Не пресмятай.

Само напиши коригирания `if`.



```
if "ivan" in users and ("admin" in roles or len(roles) >= 1):
    print("Access granted")
```

✓ Точно това имах предвид.

Тук вече не следваш готова логика, а анализираш текста:

"има поне едно от следните"

Това веднага трябва да ти светне:

Python



Run

```
admin
OR
len(roles) >= 1
```

И понеже това е второто голямо условие след:

Python



Run

```
"ivan" in users
```

естественото групиране става:

Python



Run

```
if "ivan" in users and ("admin" in roles or len(roles) >= 1):
```

Това е много по-близо до реалния coding свят.

Защото в реалността никой няма да ти напише:

Python



Run

```
A and (B or C)
```

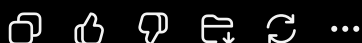
Ще ти напишат:

Потребителят трябва да е регистриран и да има или администраторски права, или активен абонамент.

И ти сам трябва да откриеш логическите блокове.

Според мен това е правилният формат за repair за теб.

Готов си за следващата задача по същия модел, но малко по-трудна — с **3 условия в един OR блок**.



не разбрах това ли е правилното решение?

Да, това е правилното решение. ✓

Нека видим защо.

Условието беше:

Достъп се дава само ако:

1. "ivan" е в users  
И
2. "има поне едно от следните:"
  - "admin" е в roles
  - "roles съдържа поне 1 роля"

Ако го преведем на логика:

```
Ivan е в users
AND
(
    admin е в roles
    OR
    len(roles) >= 1
)
```

Това става:

</> Python



▶ Run

```
if "ivan" in users and ("admin" in roles or len(roles) >= 1):
    print("Access granted")
```

Точно твоето решение.

Всъщност това е добър знак.

Преди един ден при подобна задача ти сложи скобите около едно отделно условие:

</> Python



▶ Run

```
... or (len(users) >= 2)
```

което не променяше групирането.

Сега вече си поставил скобите около **целия OR блок**:

Python



Run

```
("admin" in roles or len(roles) >= 1)
```

и това е точно целта на repair-a.

Значи можем да кажем:

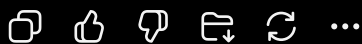
Python



Run

```
Първа repair задача: УСПЕШНА ✓
```

Дай да качим трудността с една идея нагоре следващата задача — един **AND** блок вътре в **OR** логика. Това е следващият естествен тест.



очаквам заданието

Отлично. Сега вече ще трябва сам да откриеш кой е логическият блок.

Не ти давам целевата логика.

## Repair Task 2

Python



Run

```
users = ["ivan", "maria"]
roles = ["editor"]
subscriptions = ["premium"]

# Достъп се дава ако:
#
# 1. Потребителят е в users
#
# И
#
# 2. Изпълнено е поне едно от следните:
```

```
# - има admin роля
# - има editor роля И premium абонамент
#
# Поправи само if условието.

if "ivan" in users and "admin" in roles or "editor" in roles and "premium" in subscriptions:
    print("Access granted")
```

Напиши само коригирания `if`. Нищо друго.

📄 🍴 🗑️ ↩️ ↺ ⋮

```
if ("ivan" in users and ("admin" in roles or "editor" in roles))
and "premium" in subscriptions:
    print("Access granted")
```

❌ Не е правилно.

Нека не гледаме Python, а текста.

Условието казва:

Потребителят е в users

и

Изпълнено е поне едно от следните:

- “има admin роля”
- “има editor роля И premium абонамент”

Тоест логическият блок е:

admin

ИЛИ

(editor И premium)



Твоето решение е:

<> Python



▶ Run

```
("admin" in roles or "editor" in roles)
```

което означава:

admin ИЛИ editor



а после отделно:

<> Python



▶ Run

```
and "premium" in subscriptions
```

Така ти изискваш premium и за admin случая, което не е написано в условието.

Опитай пак.

Подсказка:

Първо си представи това като:

```
USER
AND
(
    ВАРИАНТ 1
    OR
    ВАРИАНТ 2
)
```



където:

```
ВАРИАНТ 1 = ?
ВАРИАНТ 2 = ?
```



Напиши само новия `if`.



```
if "ivan" in users and ("admin" in roles or "editor" in roles
and "premium" in subscriptions):
    print("Access granted")
```

✅ Да. Това вече е правилното решение.

<> Python



▶ Run

```
if "ivan" in users and ("admin" in roles or "editor" in roles and "premium" in subscrip
```

```
print("Access granted")
```

Нека проверим спрямо текста:

## Първо условие

Python



Run

```
"ivan" in users
```

✓ Потребителят е в системата.

## Второ условие

Изпълнено е поне едно от следните:

Вариант 1:

Python



Run

```
"admin" in roles
```

ИЛИ

Вариант 2:

Python



Run

```
"editor" in roles and "premium" in subscriptions
```

Точно това си написал.

Интересното е, че тук дори без външни скоби около:

Python



Run

```
"editor" in roles and "premium" in subscriptions
```

Python пак ще го прочете правилно, защото:

Python



Run

```
and
```

има по-висок приоритет от:

</> Python

or

Run

Тоест Python автоматично вижда:

</> Python

```
"admin" in roles
or
(
    "editor" in roles
    and
    "premium" in subscriptions
)
```

Run

Това е първата задача, в която реално трябваше да преведеш текст → логически блокове, а не просто да местиш скоби.

И я реши правилно от втория опит.

Точно такъв тип гераг бих продължил още 3–5 задачи, докато започнеш автоматично да виждаш:

AND блок

OR блок

AND вътре в OR

OR вътре в AND

Run

без да мислиш дълго. Това е слабата зона, която в момента най-много си струва да успеем.

но защо and е с по висок приоритет въпреки скобите?

Много добър въпрос.

Отговорът е:

**Скобите винаги са с по-висок приоритет от всичко останало.**



В предишния пример няма конфликт между скобите и `and`.

Твоят код беше:

Python



Run

```
if "ivan" in users and ("admin" in roles or "editor" in roles and "premium" in subscrip
```

Python прави следното:

## Стъпка 1

Първо влиза в скобите:

Python



Run

```
("admin" in roles or "editor" in roles and "premium" in subscriptions)
```

Защото:

Python



Run

```
()
```

са с най-висок приоритет.

---

## Стъпка 2

ВЪТРЕ в скобите Python трябва да реши:

Python



Run

```
"admin" in roles  
or  
"editor" in roles and "premium" in subscriptions
```

Тук вече няма други скоби.

Следователно влиза стандартният приоритет:

Python



Run

```
and
```

преди

</> Python



▶ Run

```
or
```

Тоест Python го чете като:

</> Python



▶ Run

```
"admin" in roles
or
(
    "editor" in roles
    and
    "premium" in subscriptions
)
```

Мисли за това като за математика:

</> Python



▶ Run

```
2 * (3 + 4)
```

Първо:

</> Python



▶ Run

```
(3 + 4)
```

защото има скоби.

После:

</> Python



▶ Run

```
2 * 7
```

Същото е тук:

</> Python



▶ Run

```
A and (B or C and D)
```

Първо:

<> Python



▶ Run

```
(B or C and D)
```

защото е в скоби.

После вътре:

<> Python



▶ Run

```
(B or (C and D))
```

защото `and` е по-силен от `or`.

Ако искаш **OR** да е първо вътре в скобите, тогава трябва още едни скоби:

<> Python



▶ Run

```
A and ((B or C) and D)
```

Тогава Python е длъжен първо да направи:

<> Python



▶ Run

```
(B or C)
```

и чак след това:

<> Python

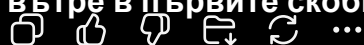


▶ Run

```
... and D
```

Това всъщност е най-честата грешка в момента при теб:

Не да забравиш скобите, а да не видиш, че **поякога трябва второ ниво скоби вътре в първите скоби**. Именно това ще тренираме в следващите gearz задачи.



хайде дай ми още една задача за gearz, явно ще имам нужда

Да. И според мен точно сега е правилният момент.

Този път ще има **две нива на групиране**.

## Repair Task 3

Python



Run

```
users = ["ivan", "maria"]
roles = ["support"]
subscriptions = ["basic"]

# Достъп се дава ако:
#
# 1. Потребителят е в users
#
# И
#
# 2. Изпълнено е едно от следните:
#
#   Вариант А:
#   - има admin роля
#
#   Вариант В:
#   - има editor И premium
#
#   Вариант С:
#   - има support И premium
#
# Поправи само if условието.

if "ivan" in users and "admin" in roles or "editor" in roles and "premium" in subscriptions
    print("Access granted")
```

Само коригирания `if`.

Не обяснявай.

Не смятай.

Само кода.



```
if "ivan" in users and ("admin" in roles or "editor" in roles
and "premium" in subscriptions) or "support" in roles and
```

```
"premium" in subscriptions:  
    print("Access granted")
```

❌ Все още не е правилно.

Това е много полезна грешка, защото показва точно къде е слабата зона.

Ти написа:

</> Python



▶ Run

```
if "ivan" in users and ("admin" in roles or "editor" in roles and "premium" in subscrip
```

Python го чете като:

</> Python



▶ Run

```
(  
    "ivan" in users  
    and  
    (  
        "admin" in roles  
        or  
        ("editor" in roles and "premium" in subscriptions)  
    )  
)  
or  
(  
    "support" in roles  
    and  
    "premium" in subscriptions  
)
```

Проблемът е, че:

</> Python



▶ Run

```
"support" in roles and "premium" in subscriptions
```

се е оказало **извън големия OR блок**.

А текстът казва:

Потребителят е в users

и

Изпълнено е едно от:

- “admin”
- “editor + premium”
- “support + premium”

Тоест:

```
USER
AND
(
    Вариант А
    OR
    Вариант В
    OR
    Вариант С
)
```



Забележи:

```
USER
AND
(
    ...
)
```



трябва да обхваща ВСИЧКИ варианти.

Подсказка:

Почни от това:

Python



Run

```
if "ivan" in users and (
    ...
):
```

и чак вътре сложи трите варианта.

Опитай още веднъж. Само `if`.



```
if "ivan" in users and ("admin" in roles or "editor" in roles
and "premium" in subscriptions or "support" in roles and
"premium" in subscriptions):
    print("Access granted")
```

✓ Да. Това вече е правилното решение.

Python



Run

```
if "ivan" in users and ("admin" in roles or "editor" in roles and "premium" in subscrip
print("Access granted")
```

Искам да отбележиш нещо много важно:

Първият ти опит беше:

Python



Run

```
USER and (A or B)
or C
```

Вторият ти опит стана:

Python



Run

```
USER and (A or B or C)
```

Това е точно поправката.

Нека го преведем на блокове:

Потребителят е в users

AND

(

admin

OR

editor + premium



```
OR
```

```
support + premium  
)
```

Това е логиката от условието.

---

Още по-важното:

Ти започваш да виждаш **границата на OR блока**.

Преди няколко задачи скобите ги поставяше около отделно условие:

```
<> Python
```



▶ Run

```
(len(users) >= 2)
```

Сега вече ги поставяш около:

```
<> Python
```



▶ Run

```
цял логически блок
```

което е истинската цел на gerair-a.

---

Според мен открихме реалната слабост:

Не е:

```
<> Python
```



▶ Run

```
and  
or
```

Не е:

```
<> Python
```



▶ Run

```
in  
len
```

А е:



</> Python



▶ Run

Къде започва и къде свършва логическият блок?

Това е много по-фина тема.

И честно казано, след последните 2 задачи вече започваш да я хващаш.

Готов си за още едно ниво нагоре: задача, в която вътре в големия OR блок има **OR** вътре в **AND**, което е най-честият източник на объркване. Това ще покаже дали repair-ът вече е стабилен или още не.

очаквам заданието

Отлично. Сега ще трябва да мислиш внимателно за това:

Кой блок принадлежи към кого?

Няма да има готова структура.

## Repair Task 4

</> Python



▶ Run

```
users = ["ivan", "maria"]
roles = ["support"]
subscriptions = ["premium"]
permissions = ["read"]

# Достъп се дава ако:
#
# 1. Потребителят е в users
#
# И
#
# 2. Изпълнено е едно от следните:
#
#   Вариант А:
#   - има admin роля
#
#   Вариант В:
#   - има editor
#   - и има поне едно от:
#       premium
```

```
#         read permission
#
#     Вариант C:
#     - има support
#     - и premium
#
# Поправи само if условието.

if "ivan" in users and "admin" in roles or "editor" in roles and "premium" in subscription:
    print("Access granted")
```

⚠ Внимавай.

Тук вече има:

```
editor
AND
(
    premium
    OR
    read
)
```



скрито вътре в един от вариантите.

Напиши само коригирания `if`. Нищо друго.

📄 👍 🔒 ⚙️ ↺ ...

тази задача е много по сложна от нивото ми, имам само няколко упражнения.. покажи ми верния отговор

Съгласен съм.

За твоето текущо ниво на Етап 7 тази задача вече беше скок от типа:

⌕ Python



▶ Run

```
A and (B or C)
```

КЪМ:

⌕ Python



▶ Run

```
A and (  
    B  
    or  
    (C and (D or E))  
    or  
    (F and G)  
)
```

Това е доста по-сложно след само няколко гераи упражнения.

Правилният отговор е:

Python



Run

```
if "ivan" in users and (  
    "admin" in roles  
    or  
    (  
        "editor" in roles  
        and  
        (  
            "premium" in subscriptions  
            or  
            "read" in permissions  
        )  
    )  
    or  
    (  
        "support" in roles  
        and  
        "premium" in subscriptions  
    )  
):  
    print("Access granted")
```

Но по-важното е **как бих го разчел**, а не как бих го написал.

Първо виждам:

```
USER  
AND  
(...)
```



Значи:

</> Python



▶ Run

```
"ivan" in users
and
(...)
```

После виждам:

има едно от следните:



Това означава:

</> Python



▶ Run

```
(
    вариант А
    or
    вариант В
    or
    вариант С
)
```

После разглеждам вариант В:

```
editor
и
поне едно от:
    premium
    read
```



Това става:

</> Python



▶ Run

```
(
    "editor" in roles
    and
    (
        "premium" in subscriptions
        or
        "read" in permissions
    )
)
```

---

Аз бих направил следното днес:

✗ НЕ още такива задачи.

✓ Още 5–10 задачи от нивото:

Python



Run

```
A and (B or C)
(A or B) and C
A and (B or C or D)
```

Защото там още изграждаш рефлексите.

Тази последната задача е по-подходяща като preview на това накъде отива логиката, отколкото като реален drill за текущото ти ниво. Това, че я усети като твърде трудна,

не искам да разводняваме roadmap-а ми за list уроците

Съгласен съм с теб.

Всъщност това е много добро наблюдение.

Като погледна roadmap-а ти:

1. List Basics
2. List Creation
3. Indexing
4. Mutation
5. Methods
6. Traversal
7. Operations (+, \*, in, len)
8. Slicing
9. Lists + Functions
10. Comprehensions
- ...

Ние вече направихме достатъчно Boolean Repair, за да разбереш основната идея:

Python



Run

```
and
or
()
```

Това беше полезно, защото се появи естествено по време на Етап 7.

Но ако продължим още 2-3 сесии само с:

</> Python

Copy

Run

A and (B or C)

ще започнем да се отдалечаваме от основната тема:

</> Python

Copy

Run

LISTS

Според мен правилният баланс е:

☒ Приключваме герар-а тук.

Запомняме:

</> Python

Copy

Run

()

контролира grouping-а.

</> Python

Copy

Run

and

има по-висок приоритет от

</> Python

Copy

Run

or

и продължаваме напред в Етап 7.

Това означава:

## Part 5 — Workflow Challenges

Но вече с фокус върху:

<> Python



▶ Run

```
+  
*  
in  
not in  
len()
```

а не върху логически пъзели.

Примерно:

- inventory systems
- order validation
- warehouse merging
- access lists
- repeated code generation
- filtering чрез membership
- collection validation

Тоест отново List Operations.

Според мен точно това е правилната посока за теб:

**не още Boolean Repair, а връщане към List roadmap-а**, като използваме наученото за `and/or`, когато се налага.

Така няма да разводняваме обучението и ще останем на Етап 7.

📄 📄 📄 📄 📄 ...



1. Урок по Python - Lis...  
PDF

за всеки случай ти давам файл изготвен от ChatGPT  
преди 2 месеца, по него вървим

Thought for a couple of seconds >

Да, виждам файла. Той потвърждава roadmap-а: ти си на **Етап 7 — List операции:**

`+`, `*`, `in`, `len()`. След него идва **Етап 8 — slicing**. 📄 1. Урок по Python - List

Затова няма да разводняваме повече с чисти boolean puzzles.

Правилният фокус оттук е:

Python



Run

```
list + list
list * n
in / not in
len()
```

с леки `if` условия само доколкото са нужни за List operations.

Следващата сесия трябва да е:

## Етап 7 — Part 5

### List Operations Workflow Practice

Не repair на `and/or`, а реални list задачи:

- комбиниране на списъци
- повторение на списъци
- проверка за елемент
- проверка за липсващ елемент
- дължина на списък
- малки workflow решения

След това, ако Part 5 мине стабилно, преминаваме към:

## Етап 8 — List slicing

Точно по файла.

Sources

очаквам задание

Перфектно. Връщаме се на Етап 7 и List Operations.

Няма скоби, няма repair на boolean логика.

Само:

Python



Run



```
+  
*  
in  
not in  
len()
```

и малко workflow мислене.

## Part 5 — List Operations Workflow Practice

### Task 1 — Team Merge

</> Python



▶ Run

```
team_a = ["Ivan", "Maria"]  
team_b = ["Petar", "Georgi"]  
  
# Създай:  
# merged_team  
  
# Ако:  
# - "Petar" съществува  
# - общият брой е 4  
  
# print("Team ready")
```

### Task 2 — Access Card Generation

</> Python



▶ Run

```
cards = ["A1", "B2"]  
  
# Създай:  
# generated_cards  
  
# чрез повторение 4 пъти  
  
# Ако:  
# - "A1" съществува  
# - дължината е 8  
  
# print("Cards generated")
```

## Task 3 — Product Validation

</> Python



▶ Run

```
products = ["mouse", "keyboard", "monitor"]

# Ако:
# - "keyboard" съществува
# - "printer" НЕ съществува
# - има точно 3 продукта

# print("Catalog valid")
```

## Task 4 — Combined Warehouse

</> Python



▶ Run

```
warehouse_a = ["monitor", "keyboard"]
warehouse_b = ["mouse"]

# Създай:
# inventory

# Ако:
# - "mouse" съществува
# - "chair" НЕ съществува
# - inventory съдържа 3 продукта

# print("Inventory ready")
```

## Task 5 — Event Registration

</> Python



▶ Run

```
registered = ["Ivan", "Maria"]
vip = ["Georgi"]

# Създай:
# all_people

# Ако:
# - "Ivan" съществува
# - "Georgi" съществува
# - общият брой е 3
```

```
# print("Event confirmed")
```

📌 Тук няма капани.

Целта е да затвърдим:

- `+`
- `*`
- `in`
- `not in`

📄 `len()` 🔗 ↻ ⋮

като оставаме стриктно в Етап 7

```
# Part 5 — List Operations Workflow Practice
```

```
# Task 1 — Team Merge
```

```
team_a = ["Ivan", "Maria"]
```

```
team_b = ["Petar", "Georgi"]
```

```
merged_team = team_a + team_b
```

```
if 'Petar' in merged_team and len(merged_team) == 4:
```

```
    print('Team ready')
```

```
# Task 2 — Access Card Generation
```

```
cards = ["A1", "B2"]
```

```
generated_cards = cards * 4
```

Show more ▾

Първо за задачите:

Python



Run

Task 1 ✓

Task 2 ✓

Task 3 ✓

Task 4 ✓

Task 5 ✓

Няма логически грешки.

Няма пропуснати условия.

Няма проблеми с `+`, `*`, `in`, `not in`, `len()`.

За другото съм напълно съгласен с теб.

Искам да направя едно важно уточнение.

Твоите **70 часа не са 70 часа List**.

Те са по-скоро:

```
List
+
Processing
+
Loop thinking
+
Index thinking
+
State reasoning
+
Workflow reasoning
```



Ако беше учил List по стандартния начин на много курсове:

Python



Run

```
append()
remove()
pop()
sort()
```

щеше да минеш темата за 3-4 часа.

Но нямаше да можеш да решаваш задачите, които решаваш днес.

Същевременно си прав, че трябва да пазим roadmap-а.

Файлът казва ясно:

```
Етап 7 → List Operations
Етап 8 → Slicing
Етап 9 → Lists + Functions
```



1. Урок по Python - List

Затова моето предложение е:

**Правило занапред**

Нова концепция може да се появява само ако:

1. естествено възниква от текущия етап;
2. не измества основната тема;
3. не води до отделна серия от уроци.

Пример:

✓ Нормално е в Етап 7 да използваме:

```
</> Python  
and
or
```

защото иначе `in` и `len()` са безполезни за реални проверки.

Но:

✗ Не е нормално да правим 3 поредни сесии само по Boolean Logic.

---

Ако следваме строго картата, аз бих казал:

## Текущ статус

Етап 7 — 90-95% завършен 

Остава:

- още малко workflow practice
- още малко самостоятелни задачи
- малко по-големи комбинации

След което:

## Етап 8 — List Slicing

И честно казано, slicing е доста по-голяма тема от това, което изглежда в документа.

Там има:

- извличане
- пропускане на елементи
- reverse slicing
- mutation чрез slicing
- slicing + loops
- slicing patterns

Така че няма опасност да останеш без материал.

Според мен за днес най-правилното е:

**още една серия от 5 задачи на Етап 7**, но малко по-самостоятелни и с по-малко подсказки.

Ако ги минеш стабилно, следващата сесия вече може да бъде:

Етап 8 — List Slicing



без чувство, че сме претупали основите.



добре, имап ли препоръчки как да подобрим roadmap-а на файла по темата List?

Да, бих направил няколко подобрения. Не бих променял основната структура, защото е добра и следва логиката на Python, но бих добавил няколко междинни етапа.

В момента файлът има:

Етап 6 → Loop  
Етап 7 → Operations  
Етап 8 → Slicing  
Етап 9 → Functions



Според мен за човек, който учи самостоятелно и иска да мисли като програмист, липсват няколко важни "моста".

1. Урок по Python - List

## Промяна №1

След Етап 6 бих добавил:

### Етап 6.5 — List Processing Patterns

Там:

<> Python



▶ Run

```
new_list = []

for x in nums:
    if condition:
        new_list.append(x)
```

- филтриране
- броене
- търсене
- събиране
- трансформация

Това е мостът към List Comprehension.

## Промяна №2

Настоящият Етап 7 бих разделил на две части.

### Етап 7А — List Operations

<> Python



▶ Run

```
+
*
in
not in
len()
```

Това вече си минал.

### Етап 7В — Workflow Logic with Lists

Тук:

<> Python



▶ Run

```
if "admin" in roles:
```

<> Python



▶ Run

```
if len(cart) > 3:
```

Python



Run

```
if "monitor" not in inventory:
```

Това е точно което правихме последните седмици.

## Промяна №3

Преди Slicing бих добавил:

### Етап 7C — Index-Based Processing

Това е много важно.

Примери:

Python



Run

```
for i in range(len(nums)):
```

Python



Run

```
nums[i]
```

Python



Run

```
nums[i+1]
```

Python



Run

```
nums[i-1]
```

Тук попадат:

- neighbor comparison
- boundary logic
- index traversal

А ти точно там направи голям напредък.

## Промяна №4

Настоящият Етап 8 бих разделил.



## Етап 8A — Slicing Basics

Python



Run

```
a[1:5]
a[:3]
a[::2]
```

## Етап 8B — Slicing Mutation

Това е отделна тема:

Python



Run

```
nums[1:3] = [100]
```

Python



Run

```
nums[:2] = []
```

Python



Run

```
nums[2:2] = [50]
```

Според мен това е една от най-трудните части на List.

## Промяна №5

Преди List Comprehension бих добавил:

## Етап 9A — Built-in Functions for Lists

Python



Run

```
sum()
max()
min()
sorted()
```

Python



Run

```
any()
```

```
all()
```

Python



Run

```
enumerate()
```

Python



Run

```
zip()
```

Защото comprehension става много по-лесен след това.

## Най-голямата промяна

Ако трябва да избира само една:

**Бих добавил отделен етап:**

List Processing Patterns



между Loop и Operations.

Това е темата, която липсва в повечето курсове и точно тя превръща:

Python



Run

```
for x in nums:
```

от синтаксис в реално мислене.

И честно казано, голяма част от твоите 70 часа всъщност са били именно този липсващ етап, затова напредъкът ти е по-дълбок от човек, който просто е минал

искам да ми дадеш цялата структура на обновени с твоите препоръки roadmap за list

Базирайки се на:

- оригиналния roadmap от PDF-а
- това, което реално си учил през последните месеци
- слабите места, които открихме

1. Урок по Python - List

- начина, по който учиш (processing-first, а не syntax-first)

бих изградил следния **List Master Roadmap v2.0**:

---

## LIST MASTER ROADMAP v2.0

### Етап 1 — What is List

#### Цел

Разбиране какво е List.

#### Теми

- Ordered collection
- Mutable
- Mixed types
- List vs variable
- List vs string
- List vs tuple

---

### Етап 2 — List Creation

#### Теми

<> Python



▶ Run

```
[]  
list()
```

#### Практика

<> Python



▶ Run

```
numbers = [1, 2, 3]  
words = ["a", "b"]  
empty = []
```

---

### Етап 3 — Indexing

#### Теми

Python



Run

```
a[0]
a[1]
a[-1]
a[-2]
```

## Практика

- първи елемент
- последен елемент
- предпоследен елемент

## Етап 4 — Mutation

### Теми

Python



Run

```
a[0] = 100
```

## Практика

- смяна по индекс
- няколко промени
- mutable концепция

## Етап 5 — List Methods

### Теми

Python



Run

```
append()
extend()
insert()
remove()
pop()
clear()
copy()
index()
count()
```

```
reverse()  
sort()
```

## Практика

CRUD върху списъци.

---

## Етап 6 — Traversal Foundations

### Теми

Python



Run

```
for x in nums
```

Python



Run

```
for i in range(len(nums))
```

Python



Run

```
enumerate()
```

## Практика

- принтиране
  - сумиране
  - броене
  - търсене
- 

## Етап 6.5 — Processing Patterns

### Теми

Python



Run

```
new_list = []
```

Python



Run

```
append()
```

като инструмент за обработка.

## Практика

- филтриране
- селекция
- трансформация
- изграждане на нов списък

## Етап 7А — List Operations

### Теми

`</> Python`



 Run

```
+  
*  
  
in  
not in  
len()
```

## Практика

- комбиниране
- повторение
- membership
- size checking

## Етап 7В — Workflow Logic With Lists

### Теми

Използване на:

`</> Python`



 Run

```
in  
not in  
len()
```

в реални проверки.

## Практика

- inventory validation
- access systems
- cart validation
- registration systems
- workflow thinking

---

## Этап 7C — Index-Based Processing

### Теми

Python



Run

```
for i in range(len(nums))
```

### Практика

Python



Run

```
nums[i]  
nums[i+1]  
nums[i-1]
```

- neighbor comparison
- rises
- drops
- pair processing
- boundary protection

---

## Этап 8A — Slicing Foundations

### Теми

Python



Run

```
a[1:5]  
a[:3]  
a[3:]  
a[::2]  
a[::-1]
```

### Практика

- извличане
- пропускане
- reverse slicing

---

## Етап 8B — Slicing Mutation

### Теми

Python



Run

```
a[1:3] = [100]
```

Python



Run

```
a[:2] = []
```

Python



Run

```
a[2:2] = [50]
```

### Практика

- replace
- delete
- insert чрез slicing

---

## Етап 9A — Built-in Functions

### Теми

Python



Run

```
sum()  
max()  
min()
```

Python



Run

```
sorted()
```



Python



Run

```
enumerate()
```

Python



Run

```
zip()
```

Python



Run

```
any()  
all()
```

## Етап 9B — Lists + Functions

### Теми

Python



Run

```
def process(items):
```

### Практика

- подаване на list
- връщане на list
- list processing functions

## Етап 10 — List Comprehension

### Теми

Python



Run

```
[x for x in nums]
```

Python



Run

```
[x for x in nums if condition]
```

### Практика

- filter
- transform
- combined processing

---

## Этап 11 — Nested Lists

### Теми

Python



Run

```
matrix = [  
    [1,2],  
    [3,4]  
]
```

### Практика

- matrix access
- nested traversal
- nested processing

---

## Этап 12 — Sorting & Processing

### Теми

Python



Run

```
sort()  
sorted()
```

Python



Run

```
key=
```

Python



Run

```
lambda
```

### Практика

- alphabetical sort
- numeric sort

- custom sort

---

## Этап 13 — Real World Combinations

### Теми

Python



Run

```
list + dict
```

Python



Run

```
list + while
```

Python



Run

```
list + function
```

### Практика

- mini systems
- business logic
- data processing

---

## Этап 14 — Common Mistakes

### Теми

Python



Run

```
IndexError
```

Python



Run

```
mutation during loop
```

Python



Run

```
shallow copy
```

<> Python



▶ Run

boundary bugs

<> Python



▶ Run

and/or grouping

## Етап 15 — Capstone Practice

### Практика

- inventory system
- order system
- registration system
- warehouse system
- SoftUni-style problems
- mini business projects

Ако трябва да оценя къде си днес.

Етап 7A — 100%

Етап 7B — ~90%

